Unit 1
Introduction to Java

History of Java - Environmental Setup - features of java - data types – variables – modifiers – keywords - Operators - Iterative, Conditional and control statement - command line arguments – string - string buffer - simple java program- enumerators-array-formatting output

1.1 History of JAVA

1. Origins of Java (1991–1995)

- Creators: Java was initiated by James Gosling, Mike Sheridan, and Patrick Naughton at Sun Microsystems in 1991.
- Original Name: Initially called Oak, named after an oak tree outside Gosling's office.
- Purpose: Designed for embedded systems (like TVs, toasters).
- Later renamed Java, inspired by Java coffee (from Indonesia), due to trademark issues with "Oak".

2. First Release (1995)

- Java 1.0 was released on May 23, 1995.
- Slogan: "Write Once, Run Anywhere" – Java programs could run on any device with the Java Virtual Machine (JVM).
- Major strength: Platform independence.

3. Growth and Versions (1996–2006)

- Java rapidly gained popularity for web development, enterprise software, and mobile applications.
- Key releases:
    - Java 1.1 (1997): Inner classes, JDBC.
    - Java 2 (1998): Renamed to Java Platform, Standard Edition (J2SE). Introduced Swing, Collections framework.
    - Java 5 (2004): Major features like Generics, Enhanced for-loop, Annotations, Enum types.
    - Java 6 (2006): Performance improvements and scripting support.

4. Oracle Acquisition (2010)

- In 2010, Oracle Corporation acquired Sun Microsystems.
- Oracle continued to maintain and develop Java, while facing legal disputes with Google over the use of Java in Android.

5. Modularization and Modern Java (2011–Present)

- Java 7 (2011): Language enhancements, try-with-resources.
- Java 8 (2014): Major update with Lambda Expressions, Streams API, Date and Time API.
- Java 9 (2017): Introduced Project Jigsaw for modularization.
- Java 10–17 (2018–2021): New features like `var` keyword, switch expressions, records, sealed classes.
- Java 21 (2023): Long-Term Support (LTS) release with modern language features and performance upgrades.

6. Current State

- Java now follows a six-month release cycle (March and September).
- Used in: Enterprise Applications, Android Development, Web Servers, Scientific Computing, and more.

Summary

Year	Milestone	Key Features
1991	Java project starts	Oak language for embedded systems
1995	Java 1.0 released	Platform independence, Applets
1998	Java 2 (J2SE)	Collections, Swing
2004	Java 5	Generics, Enums, Annotations
2010	Oracle acquires Sun	Continued Java development
2014	Java 8	Lambdas, Streams
2017	Java 9	Modular system (Jigsaw)
2023	Java 21 (LTS)	Pattern matching, Record patterns, etc.

1.2 Environmental Setup

1. Java Development Kit

The Java Development Kit (JDK) is a software development environment used for developing Java applications. It includes the Java Runtime Environment (JRE), an interpreter/loader (Java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc) and other tools needed in Java development

2. Java Runtime Environment (JRE)

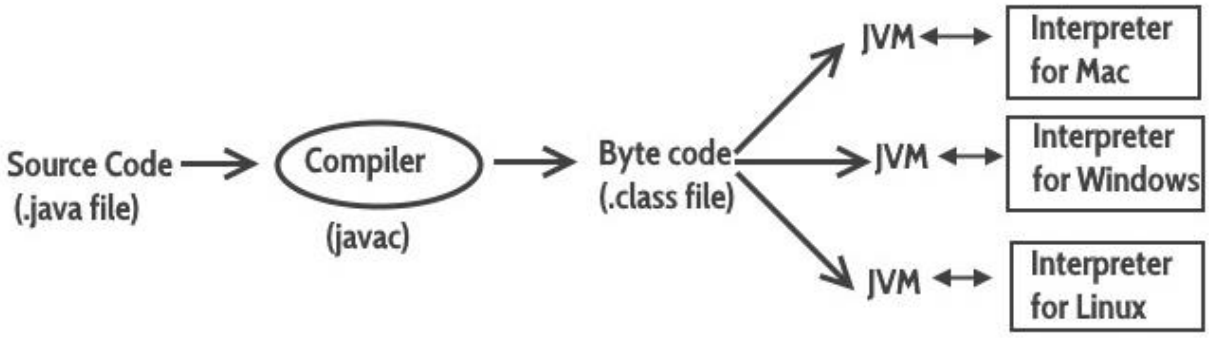
It is the most common environment available on devices to run java programs. The source Java code gets compiled and converted to Java bytecode. If you wish to run this bytecode on any platform, you require JRE. The JRE loads classes, verify access to memory, and retrieves the system resources. JRE acts as a layer on the top of the operating system.

2. Byte Code

A program written in high level language cannot be run on any machine directly. First, it needs to be translated into that particular machine language. The javac compiler does this thing, it takes java program (.java file containing source code) and translates it into machine code referred as byte code (.class file)

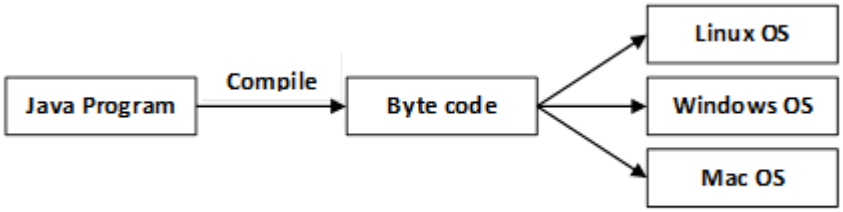
3. Java Virtual Machine (JVM)

JVM executes the byte code generated by compiler and produce output. JVM is the one that makes java platform independent.



1.3 Features of Java

1. ***Simple:*** Java is a simple language because its syntax is simple, clean, and easy to understand. Complex and ambiguous concepts of C++ are either eliminated or re-implemented in Java. For example, pointer and operator overloading are not used in Java.
2. ***Object-Oriented:*** In Java, everything is in the form of the object. It means it has some data and behavior. A program must have at least one class and object.
3. ***Robust:*** Java makes an effort to check error at run time and compile time. It uses a strong memory management system called garbage collector. Exception handling and garbage collection features make it strong.
4. ***Secure:*** Java is a secure programming language because it has no explicit pointer and programs runs in the virtual machine. Java contains a security manager that defines the access of Java classes.
5. ***Platform-Independent:*** Java provides a guarantee that code writes once and run anywhere. This byte code is platform-independent and can be run on any machine.



6. ***Portable:*** Java Byte code can be carried to any platform. No implementation-dependent features. Everything related to storage is predefined, for example, the size of primitive data types.
7. ***High Performance***: Java is an interpreted language. Java enables high performance with the use of the Just-In-Time compiler.
8. ***Distributed:*** Java also has networking facilities. It is designed for the distributed environment of the internet because it supports TCP/IP protocol. It can run over the internet. EJB and RMI are used to create a distributed system.
9. ***Multi-threaded:*** Java also supports multi-threading. It means to handle more than one job a time.

1.4 Data Types

Java is a strongly typed language. This means that every variable must have a declared type. There are eight primitive types in Java. Four of them are integer types; two are floating-point number types; one is the character type char, used for characters in the Unicode encoding (see the section on the char type), and one is a boolean type for truth values.

1. Integers

The integer types are for numbers without fractional parts. Negative values are allowed. Java provides the four integer types shown in Table.

Table 1-1. Java integer types

Type	Storage Requirement	Range (inclusive)
int	4 bytes	–2,147,483,648 to 2,147,483, 647 (just over 2 billion)
short	2 bytes	–32,768 to 32,767
long	8 bytes	–9,223,372,036,854,775,808L to 9,223,372,036,854,775,807L
byte	1 byte	–128 to 127

2. Floating-Point Types

The floating-point types denote numbers with fractional parts. There are two floating-point types, as shown in Table

Table 1-2. Floating-point types

Type	StorageRequirement	Range
float	4 bytes	approximately ±3.40282347E+38F (6–7 significant decimal digits)
double	8 bytes	approximately ±1.79769313486231570E+308 (15 significant decimal digits)

The name double refers to the fact that these numbers have twice the precision of the float type.

3. The Character Type

First, single quotes are used to denote char constants. For example, 'H' is a character. It is different from "H", a string containing a single character. Second, the char type denotes characters in the Unicode encoding scheme.

Table 1-3. Special characters

Escape Sequence	Name	Unicode Value
\b	backspace	\u0008
\t	tab	\u0009
\n	linefeed	\u000a
\r	carriage return	\u000d
\"	double quote	\u0022
\'	single quote	\u0027
\\	backslash	\u005c

4. The boolean Type

The boolean type has two values, false and true. It is used for evaluating logical conditions.

1.5 Variables

In Java, every variable has a type. You declare a variable by placing the type first, followed by the name of the variable. Here are some examples:

```
double salary;
int vacationDays;
long earthPopulation;
char yesChar;
boolean done;
```

The semicolon is necessary because a declaration is a complete Java statement.

The rules for a variable name are as follows:

A variable name must begin with a letter, and must be a sequence of letters or digits. Note that the terms ”letter” and ”digit“ are much broader in Java than in most languages. A letter is defined as 'A'–'Z', 'a'–'z', '_',

You can have multiple declarations on a single line

```
int i, j; // both are integers
```

Assignments and Initializations

After you declare a variable, you must explicitly initialize it by means of an assignment statement—you can never use the values of uninitialized variables.

You assign to a previously declared variable using the variable name on the left, an equal sign (=), and then some Java expression that has an appropriate value on the right.

```
int vacationDays; // this is a declaration
vacationDays = 12; // this is an assignment
```

Here's an example of an assignment to a character variable:

```
char yesChar;
yesChar = 'Y';
```

One nice feature of Java is the ability to both declare and initialize a variable on the same line. For example:

```
int vacationDays = 12; // this is an initialization
```

Finally, in Java you can put declarations anywhere in your code. For example, the following is valid code in Java:

```
double salary = 65000.0;
System.out.println(salary);
int vacationDays = 12; // ok to declare variable here
```

Constants

In Java, you use the keyword final to denote a constant. For example,

```
class Demo{
final double pi=3.14;
void area(int r){
//pi=3; Compile time Error
double a=pi*r*r;
System.out.println("Area of circle of radius "+r+"= "+a);
}
public static void main(String args[]){
Demo obj=new  Demo();
obj.area(5);
}  }
```

Output:

```
Area of circle of radius 5= 78.5
```

1.6 Modifiers

Modifiers in Java are keywords used to define the scope, behavior, and accessibility of classes, methods, variables, and constructors. They are categorized into Access Modifiers and Non-Access Modifiers.

1.6.1 Access Modifiers

The access modifiers or access specifiers in Java specifies the accessibility or scope of a field, method, constructor, or class. We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

There are four types of Java access modifiers:

1. ***Private:*** The access level of a private modifier is only within the class. It cannot be accessed from outside the class.

```
class A{
private A(){}                    //private constructor
private int data=40;
private void msg(){
System.out.println("Hello java");
}  }
```

```
public class Simple{
public static void main(String args[]){
A obj=new A();                          //Compile Time Error
System.out.println(obj.data); //Compile Time Error
obj.msg();                          //Compile Time Error
}
}
```

2. ***Default:*** The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.

```
//save as A.java
package pack;
class A{
void msg(){
System.out.println("Hello");
}
}
//save as B.java
package mypack;
import pack.*;
class B{
public static void main(String args[]){
A obj = new A();//Compile Time Error
obj.msg();//Compile Time Error
}
}
```

3. ***Protected:*** The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.

```
//save as A.java
package pack;
public class A{
protected void msg(){
System.out.println("Hello");
}
}
//save as B.java
package mypack;
import pack.*;
class B extends A{
public static void main(String args[]){
B obj = new B();
obj.msg();
}
}
```

Output:Hello

4. ***Public:*** The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

```
//save as A.java
package pack;
public class A{
public void msg(){
System.out.println("Hello");
}
}
```

```
//save as B.java
package mypack;
import pack.*;
class B{
public static void main(String args[]){
A obj = new A();
obj.msg();
}
}
```

Output:Hello

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

1.6.2 **Non-Access Modifiers**

These define behavioral properties like immutability, static access, or thread-safety.

1. ***static*** **-** static members are those which belongs to the class and you can access these members without instantiating the class.

Static Methods − You can create a static method by using the keyword static. Static methods can access only static fields, methods. To access static methods there is no need to instantiate the class, you can do it just using the class name

```
public class MyClass {
public static void sample(){
System.out.println("Hello");
}
public static void main(String args[]){
MyClass.sample();
}
}
```

Output

Hello

Static Fields − You can create a static field by using the keyword static. The static fields have the same value in all the instances of the class. These are created and initialized when the class is loaded for the first time. Just like static methods you can access static fields using the class name (without instantiation).

```
public class MyClass {
public static int data = 20;
public static void main(String args[]){
System.out.println(MyClass.data);
}
}
```

Output

20

Static Blocks − These are a block of codes with a static keyword. In general, these are used to initialize the static members. JVM executes static blocks before the main method at the time of class loading.

```
public class MyClass {
static{
System.out.println("Hello this is a static block");
}
public static void main(String args[]){
System.out.println("This is main method");
}
}
```

Output

```
Hello this is a static block
This is main method
```

2. ***final*** – members defined as final cannot be changed or overridden

```
final int MAX = 100;
MAX = 200;  ✗ Error: cannot change the value to final variable
```

3. ***abstract*** – abstract methods have no body, must be overridden

```
abstract class Animal {
   abstract void sound();
}
```

4. ***synchronized*** - Used for thread-safe execution

```
public synchronized void update() {
   // Only one thread at a time
}
```

1.7 Keywords

Keywords in Java can be defined as the **pre-defined** or the **reserved words** having its own importance, and each keyword has its own functionality. Since keywords are the pre-defined words used by the compiler, so they cannot be used as the variable names. If the keywords are used as the variable names, it means that we are assigning a different meaning to the keyword, which is not allowed. Java language supports nearly 50 keywords given below:

double	int	struct	break	else	long	switch	case
abstract	enum	register	typedef	char	extern	return	union
const	float	short	unsigned	continue	for	signed	void
default	goto	sizeof	volatile	do	if	static	while
public	private	protected	class	interface	extends	implements	new
this	super	try	catch	finally	throw	throws	final

1.8 Operators

Operators are special symbols (characters) that carry out operations on operands (variables and values).

1. Assignment Operator

Assignment operators are used in Java to assign values to variables. For example,

```
int age;
age = 5;
```

2. Arithmetic Operators

Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, division and modulo division.

Operator	Meaning
+	Addition (also used for string concatenation)
-	Subtraction Operator

*	Multiplication Operator
/	Division Operator
%	Remainder Operator

Example

```
class ArithmeticOperator {
public static void main(String[] args) {
      double number1 = 12.5, number2 = 3.5, result;
      // Using addition operator
      result = number1 + number2;
System.out.println("number1 + number2 = " + result);
      // Using subtraction operator
      result = number1 - number2;
System.out.println("number1 - number2 = " + result);
      // Using multiplication operator
      result = number1 * number2;
System.out.println("number1 * number2 = " + result);
      // Using division operator
      result = number1 / number2;
System.out.println("number1 / number2 = " + result);
      // Using remainder operator
      result = number1 % number2;
System.out.println("number1 % number2 = " + result);
   }
}
```

Output

```
number1 + number2 = 16.0
number1 - number2 = 9.0
number1 * number2 = 43.75
number1 / number2 = 3.5714285714285716
number1 % number2 = 2.0
```

3. Unary Operators

The unary operator performs operations on only one operand.

Operator	Meaning
+	Unary plus (not necessary to use since numbers are positive without using it)
-	Unary minus: inverts the sign of an expression
++	Increment operator: increments value by 1
--	decrement operator: decrements value by 1
!	Logical complement operator: inverts the value of a boolean

Example

```
class UnaryOperator {
   public static void main(String[] args) {
      double number = 5.2, resultNumber;
boolean flag = false;
System.out.println("+number = " + +number);
System.out.println("-number = " + -number);
      // ++number is equivalent to number = number + 1
System.out.println("number = " + ++number);
      // -- number is equivalent to number = number - 1
System.out.println("number = " + --number);
System.out.println("!flag = " + !flag);
   }
}
```

Output

```
+number = 5.2
-number = -5.2
number = 6.2
number = 5.2
!flag = true
```

4. Increment and Decrement Operator

You can also use ++ and -- operator as both prefix and postfix in Java. The ++ operator increases value by 1 and -- operator decreases the value by 1.

Example

```
int myInt = 5;
++myInt   // myInt becomes 6
myInt++   // myInt becomes 7
--myInt   // myInt becomes 6
myInt--   // myInt becomes 5
```

5. Equality and Relational Operators

The equality and relational operators determine the relationship between the two operands. It checks if an operand is greater than, less than, equal to, not equal to and so on. Depending on the relationship, it is evaluated to either true or false.

Operator	Description	Example
==	equal to	5 == 3 is evaluated to false
!=	not equal to	5 != 3 is evaluated to true
>	greater than	5 > 3 is evaluated to true
<	less than	5 < 3 is evaluated to false
>=	greater than or equal to	5 >= 5 is evaluated to true
<=	less than or equal to	5 <= 5 is evaluated to true

6. instanceof Operator

In addition to relational operators, there is also a type comparison operator instanceof which compares an

Example

```
class instanceofOperator {
   public static void main(String[] args) {
      String test = "asdf";
boolean result;
      result = test instanceof String;
System.out.println("Is test an object of String? " + result);
   }
}
```

Output

```
Is test an object of String? true
```

7. Logical Operators

The logical operators || (conditional-OR) and && (conditional-AND) operate on boolean expressions. Here's how they work.

Operator	Description	Example
||	conditional-OR: true if either of the boolean expression is true	false || true is evaluated to true
&&	conditional-AND: true if all boolean expressions are true	false && true is evaluated to false

8. Ternary Operator

The conditional operator or ternary operator ?: is shorthand for the if-then-else statement. The syntax of the conditional operator is:

variable = Expression ? expression1 : expression2

Here's how it works.

- If the Expression is true, expression1 is assigned to the variable.
- If the Expression is false, expression2 is assigned to the variable.

Example

```
class ConditionalOperator {
   public static void main(String[] args) {
      int februaryDays = 29;
      String result;
         result =  (februaryDays == 28) ? "Not a leap year" : "Leap year";
System.out.println(result);    }}
```

Output:

```
Leap year
```

9. Bitwise and Bit Shift Operators

To perform bitwise and bit shift operators in Java, these operators are used.

Operator	Description
~	Bitwise Complement
<<	Left Shift
>>	Right Shift
>>>	Unsigned Right Shift
&	Bitwise AND
^	Bitwise exclusive OR
|	Bitwise inclusive OR

These operators are not commonly used. To learn more, visit Java bitwise and bit shift operators.

10. More Assignment Operators

We have only discussed one assignment operator = at the beginning of the article. Except for this operator, there are quite a few assignment operators that hels us to write cleaner code.

Operator	Example	Equivalent to
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5
<<=	x <<= 5	x = x << 5
>>=	x >>= 5	x = x >> 5
&=	x &= 5	x = x & 5
^=	x ^= 5	x = x ^ 5
|=	x |= 5	x = x | 5

Operator precedence

Operators	Associativity
[] . () (method call)	left to right
! ~ ++ -- + (unary) – (unary) () (cast) new	right to left
* / %	left to right
+ -	left to right
<<>>>>>	left to right
<<= >>= instanceof	left to right

Operators	Associativity
== !=	left to right
&	left to right
^	left to right
|	left to right
&&	left to right
||	left to right
?:	left to right
= += -= *= /= %= &= |= ^= <<= >>= >>>=	right to left

1.9 Control Flow

In Java there are three types of control flow statements:

1. Conditional/ Decision making statements: if, if else, nested if else, switch case
2. Iterative/ Looping statements: while, for, do-while
3. Branching statements: return, break, continue

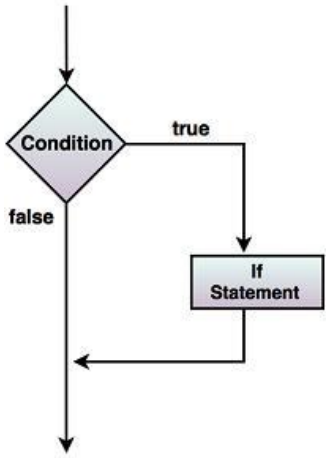
1. if statement

It instructs the program to execute only certain part of the code if the condition is true.
If boolean value is true then statement1 will be executed.

Syntax:

```
if (boolean expr)
{
    statement 1:      //expr true
}
statement 2;
```

Flow Diagram



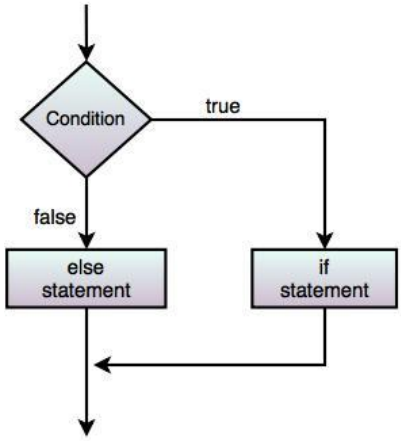
2. if else statement

In this case, when if clause is false then else part will be executed.

Syntax:

```
if (boolean expr)
{
    statement 1;      //expr true
}
else
{
    statement 2;      //expr false
}
```

Flow Diagram



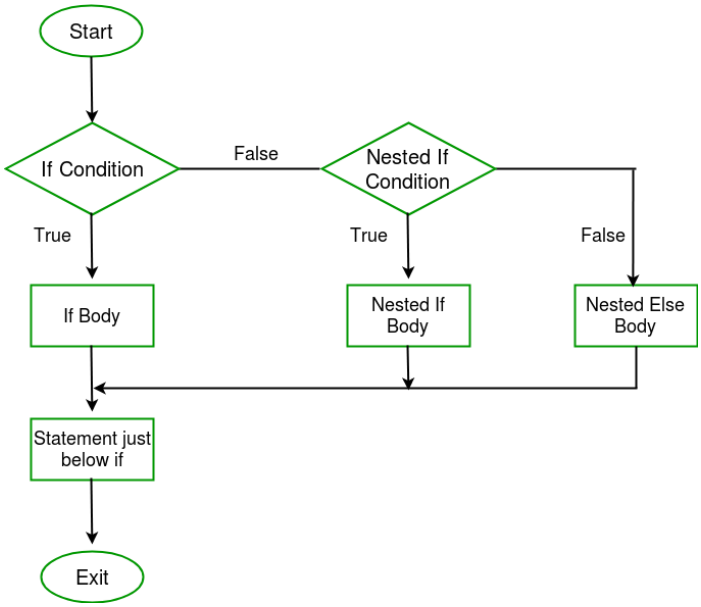
3. Nested if else

you can use one if or else if statement inside another if or else if statement.

Syntax:

```
if (boolean expr1)
{
    statement 1;      // expr1 true
}
else if (boolean expr2)
{
    Statement 2;      // expr 1 false and expr 2 true
}
else
{
    statement 3;      // expr 2 false
}
```

Flow Diagram



4. Switch Case Statement

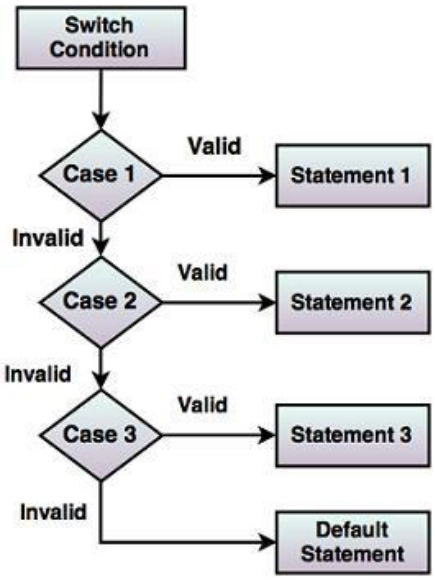
It is type of selection mechanism that provides multi-way selection.

Syntax

```
switch(expression) {
  case value :
    // Statements
    break; // optional
    case value :
    // Statements
    break; // optional
```

```
    // You can have any number of case statements.
  default : // Optional
    // Statements
}
```

Flow Diagram



5. While statement

While loop is used for continually executing a block of statement.It repeats the statement while a given condition is true.

Syntax:

```
while (expresison)
{
    Statements ;
}
```

Program to print the Fibonacci series using while loop

```
class FiboTest
{
    public static void main(String args[])
    {
        int a = 0;
        int max = 50;
        int fib = 1;
System.out.print("Fibonacci series is: "+a);
        while ( fib <= max )
        {
System.out.print("  "+fib);
            fib = a+fib;
            a = fib;
        }
    }
}
```

Output:

```
Fibonacci series is: 0  1  1  2  3  5  8  13  21  34
```

6. for loop

The for loop also repeats until given condition is met.

Syntax:

```
for (initialization; condition; increment/decrement)
{
    statements ;
}
```

Program to print even numbers between the given ranges using for loop.

```
class EvenNumber
{
    public static void main(String args[])
    {
        int n;
System.out.print("Even number are: ");
        for( n = 0; n <= 20; ++n)
        {
            if (n % 2 == 0)
System.out.print("  "+n);
        }
    }}
```

Output:

```
Even number are:  0  2  4  6  8  10  12  14  16  18  20
```

7. do-while loop

The statement inside the do block executes at least once until the condition of while is either true or false.

Syntax:

```
do
{
    Statements ;
}
while (condition) ;
```

Simple program for do-while loop

```
class DoWhile
{
    public static void main(String args[])
    {
        int a = 1;
        do
        {
System.out.println("Value of a : "+a);
            ++a;
        }
        while (a<10);
    }
}
```

Output:

```
Value of a : 1
Value of a : 2
Value of a : 3
Value of a : 4
Value of a : 5
Value of a : 6
Value of a : 7
Value of a : 8
Value of a : 9
```

8. Break Statement

Break statement is basically used for exit from loop.In case of break it goes completely out of all the nested loops.

Simple program for break statement

```
class BreakDemo
{
    public static void main(String args[])
    {
```

```
        int arr[] = {5,10,15,20,25,30};
        int n ;
        int search = 15;
boolean b = false;
        for(n = 0; n <arr.length; n++)
        {
            if(arr[n] == search)
            {
                b = true;
                break;
            }
        }
        if(b)
System.out.println("Number found " +search+" at index of  "+n);
        else
System.out.println("Element is not found");
    }
}
```

Output:

Number found 15 at index of 2

9. Continue Statement

Continue statement is used to skip the current iteration of while, for or do-while loop.It causes the immediate jump to next iteration of loop.

Program for printing odd and even number is different columns.

```
        class ContinueDemo
        {
            public static void main(String []args)
            {
                for (int j = 1; j <= 10; j++)
                {
        System.out.print(j+"  ");
                    if (j % 2 != 0)
                        continue;
        System.out.println("  ");
                }     }         }
```

Output:

1 2
3 4
5 6
7 8
9 10

1.10 Command-Line Arguments

The command-line arguments in Java allow us to pass arguments during the execution of the program. As the name suggests arguments are passed through the command line. These are passed to the main() method as a String array.

Example:

```
public class PrintArgs {
    public static void main(String[] args) {
        System.out.println("Number of arguments: " + args.length);
        for (int i = 0; i < args.length; i++) {
            System.out.println("Argument " + i + ": " + args[i]);
        }
    }
}
```

Run the code as: java PrintArgs Hello World 123

Output:

Argument 1: Hello
Argument 2: World
Argument 3: 123

Example: Add Two Numbers Using Command-Line Arguments

```
public class AddNumbers {
   public static void main(String[] args) {
      // Check if two arguments are provided
      if (args.length < 2) {
         System.out.println("Please provide two numbers as command-line arguments.");
         return;
      }
      // Convert String arguments to integers
      int num1 = Integer.parseInt(args[0]);
      int num2 = Integer.parseInt(args[1]);
      // Add the numbers
      int sum = num1 + num2;
      // Display the result
      System.out.println("The sum of " + num1 + " and " + num2 + " is: " + sum);
   }
}
```

Run the code as: java AddNumbers 456 123

Output: The sum of 456 and 123 is: 579

1.11 Strings

In Java, a string is a sequence of characters. For example, "hello" is a string containing a sequence of characters 'h', 'e', 'l', 'l', and 'o'.

1.11.1 Java String Methods

Java String provides various methods that allow us to perform different string operations. Here are some of the commonly used string methods.

Methods	Description
S1.concat(S2)	joins the two strings together
S1.equals(S2)	compares the value of two strings
S1.compareTo(S2)	If S1<S2, positive value is returned If S1>S2, negative value is returned If S1=S2, zero is returned
S1.charAt(pos)	returns the character present in the specified location
S1.getBytes()	converts the string to an array of bytes
S1.indexOf('c')	returns the position of the specified character in the string
S1.indexOf('c',n)	returns the position of the specified character in the string after n^{th} position
S1.length()	returns the size of the specified string
S1.replace()	replaces the specified old character with the specified new character
S1.substring()	returns the substring of the string
S1.split()	breaks the string into an array of strings
S1.toLowerCase()	converts the string to lowercase
S1.toUpperCase()	converts the string to uppercase
S1.valueOf()	returns the string representation of the specified data
S1.equalsIgnoreCase(S2)	If S1and S2 are equal, it returns true

Example 1: find string's length

```
class Main {
public static void main(String[] args) {
String greet = "Hello! World";
System.out.println("The string is: " + greet);
```

```
System.out.println("The length of the string: " + greet.length());
}
}
```

Output

```
The string is: Hello! World
The length of the string: 12
```

Example 2: join two strings using concat()

```
class Main {
public static void main(String[] args) {
String greet = "Hello! ";
System.out.println("First String: " + greet);
String name = "World";
System.out.println("Second String: " + name);
String joinedString = greet.concat(name);
System.out.println("Joined String: " + joinedString);
}
}
```

Output

```
First String: Hello!
Second String: World
Joined String: Hello! World
```

Example 3: join strings using + operator

```
class Main {
public static void main(String[] args) {
String greet = "Hello! ";
System.out.println("First String: " + greet);
String name = "World";
System.out.println("Second String: " + name);
String joinedString = greet + name;
System.out.println("Joined String: " + joinedString);
}
}
```

Output

```
First String: Hello!
Second String: World
Joined String: Hello! World
```

Example 4: compare two strings

```
class Main {
public static void main(String[] args) {
String greet ="Hello!";
String name1=" World";
String name2=" World";
System.out.println("The  1st string is: "+greet);
System.out.println("The  2nd string is: "+name1);
System.out.println("The  3rd string is: "+name2);
System.out.println("string1 and string2 are equal: "+greet.equals(name1));
System.out.println("string2 and string3 are equal: "+name2.equals(name1));
}}
```

Output

```
The  1st string is: Hello!
The  2nd string is:  World
The  3rd string is:  World
string1 and string2 are equal: false
string2 and string3 are equal: true
```

Example 5: get characters from a string

```
class Main {
public static void main(String[] args) {
String greet = "Hello! World";
System.out.println("The string is: " + greet);
System.out.println("The character at 3: " + greet.charAt(3));
System.out.println("The character at 7: " + greet.charAt(7));
}}
```

Output

```
The string is: Hello! World
The character at 3: l
The character at 7: W
```

Example 6: Strings other methods

```
class Main {
  public static void main(String[] args) {
    // create string using the new keyword
    String example = new String("Hello! World");
    // returns the substring World
    System.out.println("Using the subString(): " + example.substring(7));
    // converts the string to lowercase
    System.out.println("Using the toLowerCase(): " + example.toLowerCase());
    // converts the string to uppercase
    System.out.println("Using the toUpperCase(): " + example.toUpperCase());
    // replaces the character '!' with 'o'
    System.out.println("Using the replace(): " + example.replace('!', 'o'));
  }}
```

Output

```
Using the subString(): World
Using the toLowerCase(): hello! world
Using the toUpperCase(): HELLO! WORLD
Using the replace(): Helloo World
```

1.11.2 StringBuffer class

- used to create mutable (modifiable) String objects.
- same as String class except it is mutable i.e. it can be changed.

Constructors of StringBuffer Class

Constructor	Description
StringBuffer()	It creates an empty String buffer with the initial capacity of 16.
StringBuffer(String str)	It creates a String buffer with the specified string.
StringBuffer(int capacity)	It creates an empty String buffer with the specified capacity as length.

Methods of StringBuffer Class

Method	Description
append(String s)	It is used to append the specified string with this string. The append() method is overloaded like append(char), append(boolean), append(int), append(float), append(double) etc.
insert(int offset, String s)	It is used to insert the specified string with this string at the specified position. The insert() method is overloaded like insert(int, char), insert(int, boolean), insert(int, int), insert(int, float), insert(int, double) etc.
replace(int startIndex, int endIndex, String str)	It is used to replace the string from specified startIndex and endIndex.
delete(int startIndex, int endIndex)	It is used to delete the string from specified startIndex and endIndex.
reverse()	is used to reverse the string.

capacity()	It is used to return the current capacity. The default capacity of the buffer is 16.
ensureCapacity(int minimumCapacity)	It is used to ensure the capacity at least equal to the given minimum. If the number of character increases from its current capacity, it increases the capacity by (oldcapacity*2)+2. For example if your current capacity is 16, it will be (16*2)+2=34.
charAt(int index)	It is used to return the character at the specified position.
length()	It is used to return the length of the string i.e. total number of characters.
substring(int beginIndex)	It is used to return the substring from the specified beginIndex.
substring(int beginIndex, int endIndex)	It is used to return the substring from the specified beginIndex and endIndex.

Example:

```
class StringBufferExample{
public static void main(String args[]){
StringBuffer sb=new StringBuffer("Hello ");
System.out.println(sb);
sb.append("Java");
System.out.println(sb);
sb.insert(10," Programs");
System.out.println(sb);
sb.replace(1,3,"Java");
System.out.println(sb);
sb.delete(1,3);
System.out.println(sb);
sb.reverse();
System.out.println(sb);
StringBuffer sb1=new StringBuffer("Hello");
System.out.println(sb1);
System.out.println(sb1.capacity());
sb1.append(" java is my favourite language");
System.out.println(sb1);
sb1.ensureCapacity(10);
System.out.println(sb1.capacity());
System.out.println(sb1.charAt(3));
System.out.println(sb1.length());
System.out.println(sb1.substring(14));
System.out.println(sb1.substring(14,26));
} }
```

Output:

```
Hello
Hello Java
Hello Java Programs
HJavalo Java Programs
Hvalo Java Programs
smargorP avaJ olavH
Hello
21
Hello java is my favourite language
44
l
35
my favourite language
my favourite
```

1.12 Enumerators

Enums (short for **enumerations**) are a special data type in Java used to define **a fixed set of constants**. They make your code more readable and less error-prone when you have to work with fixed values like days of the week, directions, colors, etc.

An enum can, just like a class, have attributes and methods. The only difference is that enum constants are public, static and final (unchangeable - cannot be overridden).

An enum cannot be used to create objects, and it cannot extend other classes (but it can implement interfaces).

Example:

```
enum Day {
   SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY
}
public class EnumExample {
   public static void main(String[] args) {
      Day today = Day.TUESDAY;
      System.out.println("Today is: " + today);
      switch (today) {
         case MONDAY:
            System.out.println("Start of the work week!");
            break;
         case FRIDAY:
            System.out.println("End of the work week!");
            break;
         case SUNDAY:
            System.out.println("It's a holiday!");
            break;
         default:
            System.out.println("Midweek day.");
      }    }          }
```

1.13 Arrays

An array is a container object that contains the similar type of data. It can hold both primitive and object type data.Each item in an array is called an element and each element is accessed by its numeric index.

Features of Arrays

- Retrieve and sort the data easily.
- Arrays are created during runtime.
- Arrays can hold the reference variables of other objects.
- Arrays are fixed in size.

Types of Arrays

1. Single dimensional arrays
2. Multidimensional arrays

1. Single Dimensional Arrays

Declaration of Single Dimensional Arrays

```
int[ ] arr or int [ ]arr or int arr[ ];
byte[ ] arr;
short[ ] arr;
long[ ] arr;
char[ ] chr;
String[ ] strect;
```

Creating, Initializing and Accessing an Array

```
int arr = new int[10];    // creation of integer array of size 10
```

An example of single dimensional array in Java.

```
public class ArrayDemo{
public static void main(String args[]){
int arr[] = new int[6];
arr[]={2, 4, 5, 7, 9, 10};
for(int i = 0; i<6; i++)
System.out.println("arr["+i+"] = "+arr[i]);
} }
```

Output:

```
arr[0] = 2
arr[1] = 4
arr[2] = 5
arr[3] = 7
arr[4] = 9
arr[5] = 10
```

2. Multidimensional Arrays

In multidimensional array data is stored in form of rows and columns.

Declaration of Multidimensional Arrays

int arr[ ][ ], or int[ ][ ] arr, or int [ ][ ]arr.
char arr[ ][ ] or char[ ][ ] arr or char [ ][ ]arr
double arr[ ][ ] or double[ ][ ] arr or double [ ][ ]arr.

A sample program to create a multidimensional array.

```
public class MultiDimeDemo
{
public static void main(String args[])
{
int arr[] [] = {{9,8,7},{6,5,4},{3,2,1}};
for(int i = 0; i< 3; i++)       {
for(int j = 0; j < 3; j++)     {
System.out.print(arr[i][j]+" ");
}
System.out.println();
}
}
}
```

Output:

```
9 8 7
6 5 4
2 2 1
```

1.14 Formatting Output

Java provides several ways to format output, especially for displaying numbers, strings, and dates in a readable and organized way.

1. System.out.printf() – Formatted Output

This works like printf in C/C++.

Basic Syntax:

```
System.out.printf("format string", arguments);
```

Example:

```
public class FormatExample
{
  public static void main(String[] args)
{
        String name = "Alice";
        int age = 25;
        double score = 89.4567;
        System.out.printf("Name: %s\n", name);              // String
        System.out.printf("Age: %d\n", age);               // Integer
        System.out.printf("Score: %.2f\n", score);          // Floating point with 2 decimals
        System.out.printf("Padded Score: %10.2f\n", score);    // Padded to 10 characters
    }
}
```

Output:

```
Name: Alice
Age: 25
Score: 89.46
Padded Score:      89.46
```

2. Format Specifiers

Format	Use for	Example
%d	Integer	25
%f	Float/Double	12.34
%.2f	Float with 2 decimals	12.34
%s	String	"Hello"
%c	Character	'A'
%n	New line (like \n)	
%10d	Right-aligned int (width 10)	
%-10s	Left-aligned string (width 10)	

3. Using String.format() (to store formatted strings)

```
String result = String.format("Hello %s, you scored %.1f%%", "Bob", 95.6);
System.out.println(result);
```

4. Number Formatting with DecimalFormat

```
import java.text.DecimalFormat;
public class DecimalFormatExample {
   public static void main(String[] args) {
      double num = 1234.56789;
      DecimalFormat df = new DecimalFormat("#.##");
      System.out.println("Formatted: " + df.format(num));  // Output: 1234.57
   }
}
```

5. Padding and Alignment Example

```
System.out.printf("%-10s %5d %8.2f%n", "Alice", 25, 95.75);
System.out.printf("%-10s %5d %8.2f%n", "Bob", 30, 88.9);
```

Output:

```
Alice       25    95.75
Bob         30    88.90
```